JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Class SelectionKey

java.lang.Object
    java.nio.channels.SelectionKey

Direct Known Subclasses:

AbstractSelectionKey

```
public abstract class SelectionKey
extends Object
```

A token representing the registration of a `SelectableChannel` with a `Selector`.

A selection key is created each time a channel is registered with a selector. A key remains valid until it is *cancelled* by invoking its `cancel` method, by closing its channel, or by closing its selector. Cancelling a key does not immediately remove it from its selector; it is instead added to the selector's *cancelled-key set* for removal during the next selection operation. The validity of a key may be tested by invoking its `isValid` method.

A selection key contains two *operation sets* represented as integer values. Each bit of an operation set denotes a category of selectable operations that are supported by the key's channel.

- The *interest set* determines which operation categories will be tested for readiness the next time one of the selector's selection methods is invoked. The interest set is initialized with the value given when the key is created; it may later be changed via the `interestOps(int)` method.
- The *ready set* identifies the operation categories for which the key's channel has been detected to be ready by the key's selector. The ready set is initialized to zero when the key is created; it may later be updated by the selector during a selection operation, but it cannot be updated directly.

That a selection key's ready set indicates that its channel is ready for some operation category is a hint, but not a guarantee, that an operation in such a category may be performed by a thread without causing the thread to block. A ready set is most likely to be accurate immediately after the completion of a selection operation. It is likely to be made inaccurate by external events and by I/O operations that are invoked upon the corresponding channel.

This class defines all known operation-set bits, but precisely which bits are supported by a given channel depends upon the type of the channel. Each subclass of `SelectableChannel` defines an `validOps()` method which returns a set identifying just those operations that are supported by the channel. An attempt to set or test an operation-set bit that is not supported by a key's channel will result in an appropriate run-time exception.

It is often necessary to associate some application-specific data with a selection key, for example an object that represents the state of a higher-level protocol and handles readiness notifications in order to implement that protocol. Selection keys therefore support the *attachment* of a single arbitrary object to a key. An object can be attached via the `attach` method and then later retrieved via the `attachment` method.

Selection keys are safe for use by multiple concurrent threads. The operations of reading and writing the interest set will, in general, be synchronized with certain operations of the selector. Exactly how this synchronization is performed is implementation-dependent: In a naive implementation, reading or writing the interest set may block indefinitely if a selection operation is already in progress; in a high-performance implementation, reading or writing the interest set may block briefly, if at all. In any case, a selection operation will always use the interest-set value that was current at the moment that the operation began.

Since:

1.4

See Also:

`SelectableChannel`, `Selector`

Field Summary

Fields

Modifier and Type	Field	Description
static int	**OP_ACCEPT**	Operation-set bit for socket-accept operations.
static int	**OP_CONNECT**	Operation-set bit for socket-connect operations.
static int	**OP_READ**	Operation-set bit for read operations.

static int	OP_WRITE	Operation-set bit for write operations.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	SelectionKey()	Constructs an instance of this class.

Method Summary

Abstract Methods | **Concrete Methods**

Modifier and Type	Method	Description
Object	attach(Object ob)	Attaches the given object to this key.
Object	attachment()	Retrieves the current attachment.
abstract void	cancel()	Requests that the registration of this key's channel with its selector be cancelled.
abstract SelectableChannel	channel()	Returns the channel for which this key was created.
abstract int	interestOps()	Retrieves this key's interest set.
abstract SelectionKey	interestOps(int ops)	Sets this key's interest set to the given value.
boolean	isAcceptable()	Tests whether this key's channel is ready to accept a

		new socket connection.
boolean	**isConnectable**()	Tests whether this key's channel has either finished, or failed to finish, its socket-connection operation.
boolean	**isReadable**()	Tests whether this key's channel is ready for reading.
abstract boolean	**isValid**()	Tells whether or not this key is valid.
boolean	**isWritable**()	Tests whether this key's channel is ready for writing.
abstract int	**readyOps**()	Retrieves this key's ready-operation set.
abstract **Selector**	**selector**()	Returns the selector for which this key was created.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

OP_READ

```
public static final int OP_READ
```

Operation-set bit for read operations.

Suppose that a selection key's interest set contains `OP_READ` at the start of a selection operation. If the selector detects that the corresponding channel is ready for reading, has reached end-of-stream, has been remotely shut down

for further reading, or has an error pending, then it will add OP_READ to the key's ready-operation set and add the key to its selected-key set.

See Also:

Constant Field Values

OP_WRITE

```
public static final int OP_WRITE
```

Operation-set bit for write operations.

Suppose that a selection key's interest set contains OP_WRITE at the start of a selection operation. If the selector detects that the corresponding channel is ready for writing, has been remotely shut down for further writing, or has an error pending, then it will add OP_WRITE to the key's ready set and add the key to its selected-key set.

See Also:

Constant Field Values

OP_CONNECT

```
public static final int OP_CONNECT
```

Operation-set bit for socket-connect operations.

Suppose that a selection key's interest set contains OP_CONNECT at the start of a selection operation. If the selector detects that the corresponding socket channel is ready to complete its connection sequence, or has an error pending, then it will add OP_CONNECT to the key's ready set and add the key to its selected-key set.

See Also:

Constant Field Values

OP_ACCEPT

```
public static final int OP_ACCEPT
```

Operation-set bit for socket-accept operations.

Suppose that a selection key's interest set contains OP_ACCEPT at the start of a selection operation. If the selector detects that the corresponding server-socket channel is ready to accept another connection, or has an error pending, then it will add OP_ACCEPT to the key's ready set and add the key to its selected-key set.

See Also:

Constant Field Values

Constructor Detail

SelectionKey

```
protected SelectionKey()
```

Constructs an instance of this class.

Method Detail

channel

```
public abstract SelectableChannel channel()
```

Returns the channel for which this key was created. This method will continue to return the channel even after the key is cancelled.

Returns:

This key's channel

selector

```
public abstract Selector selector()
```

Returns the selector for which this key was created. This method will continue to return the selector even after the key is cancelled.

Returns:

This key's selector

isValid

```
public abstract boolean isValid()
```

Tells whether or not this key is valid.

A key is valid upon creation and remains so until it is cancelled, its channel is closed, or its selector is closed.

Returns:

`true if, and only if, this key is valid`

cancel

```
public abstract void cancel()
```

Requests that the registration of this key's channel with its selector be cancelled. Upon return the key will be invalid and will have been added to its selector's cancelled-key set. The key will be removed from all of the selector's key sets during the next selection operation.

If this key has already been cancelled then invoking this method has no effect. Once cancelled, a key remains forever invalid.

This method may be invoked at any time. It synchronizes on the selector's cancelled-key set, and therefore may block briefly if invoked concurrently with a cancellation or selection operation involving the same selector.

interestOps

```
public abstract int interestOps()
```

Retrieves this key's interest set.

It is guaranteed that the returned set will only contain operation bits that are valid for this key's channel.

This method may be invoked at any time. Whether or not it blocks, and for how long, is implementation-dependent.

Returns:

`This key's interest set`

Throws:

`CancelledKeyException - If this key has been cancelled`

interestOps

```
public abstract SelectionKey interestOps(int ops)
```

Sets this key's interest set to the given value.

This method may be invoked at any time. Whether or not it blocks, and for how long, is implementation-dependent.

Parameters:

ops - The new interest set

Returns:

This selection key

Throws:

IllegalArgumentException - If a bit in the set does not correspond to an operation that is supported by this key's channel, that is, if (ops & ~channel().validOps()) != 0

CancelledKeyException - If this key has been cancelled

readyOps

```
public abstract int readyOps()
```

Retrieves this key's ready-operation set.

It is guaranteed that the returned set will only contain operation bits that are valid for this key's channel.

Returns:

This key's ready-operation set

Throws:

CancelledKeyException - If this key has been cancelled

isReadable

```
public final boolean isReadable()
```

Tests whether this key's channel is ready for reading.

An invocation of this method of the form `k.isReadable()` behaves in exactly the same way as the expression

```
k.readyOps() & OP_READ != 0
```

If this key's channel does not support read operations then this method always returns `false`.

Returns:

true if, and only if, readyOps() & OP_READ is nonzero

Throws:

CancelledKeyException - If this key has been cancelled

isWritable

```
public final boolean isWritable()
```

Tests whether this key's channel is ready for writing.

An invocation of this method of the form `k.isWritable()` behaves in exactly the same way as the expression

```
k.readyOps() & OP_WRITE != 0
```

If this key's channel does not support write operations then this method always returns `false`.

Returns:

`true if, and only if, readyOps() & OP_WRITE is nonzero`

Throws:

`CancelledKeyException - If this key has been cancelled`

isConnectable

```
public final boolean isConnectable()
```

Tests whether this key's channel has either finished, or failed to finish, its socket-connection operation.

An invocation of this method of the form `k.isConnectable()` behaves in exactly the same way as the expression

```
k.readyOps() & OP_CONNECT != 0
```

If this key's channel does not support socket-connect operations then this method always returns `false`.

Returns:

`true if, and only if, readyOps() & OP_CONNECT is nonzero`

Throws:

`CancelledKeyException - If this key has been cancelled`

isAcceptable

```
public final boolean isAcceptable()
```

Tests whether this key's channel is ready to accept a new socket connection.

An invocation of this method of the form `k.isAcceptable()` behaves in exactly the same way as the expression

```
k.readyOps() & OP_ACCEPT != 0
```

If this key's channel does not support socket-accept operations then this method always returns `false`.

Returns:

`true if, and only if, readyOps() & OP_ACCEPT is nonzero`

Throws:

`CancelledKeyException - If this key has been cancelled`

attach

```
public final Object attach(Object ob)
```

Attaches the given object to this key.

An attached object may later be retrieved via the `attachment` method. Only one object may be attached at a time; invoking this method causes any previous attachment to be discarded. The current attachment may be discarded by attaching `null`.

Parameters:

`ob - The object to be attached; may be null`

Returns:

`The previously-attached object, if any, otherwise null`

attachment

```
public final Object attachment()
```

Retrieves the current attachment.

Returns:

C

```
The object currently attached to this key, or null if there is no
attachment
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.